

M3-M5：对象、地址、指针、数组与自增

CSC3002 基础讲义 · 第 1 册 · 从 M2 的函数与引用继续

这一册只围绕一个问题：每个表达式到底在访问哪个对象，又改变了谁？ 学完应能区分 `x`、`&x`、`p`、`*p`、`&p`，解释数组地址步长，并独立手推四种“指针与 ++”表达式。

路线：对象与存储 → 地址 → 指针变量 → 解引用 → 指针与引用 → 数组和边界 → 指针算术 → 普通自增 → 四种组合 → 综合手推。先把单个机制讲清，最后再组合。

本册以 L6 的对象、指针、数组与指针运算为主，结合 L2 的自增和 E22 原题重排教学顺序，采用 C++17。先建立对象关系，再进入 Q31/Q32；地址图使用题设或明确的示意地址。片段若使用 `std::cout`，默认已包含 `<iostream>` 并位于函数体内。

阅读依据与课程定位

这是对实际课程材料重新核对后的修订版，核对日期为 2026-10-08。L 后的数字是课件代号；“第几张”按文件实际幻灯片顺序计数，从封面开始，不沿用旧模板页脚。E22 页码为 PDF 页码。

- L2: 2-CSC3002_Lecture Notes Functions and Libraries Part 1(2).pptx
- L6: 6-CSC3002_Pointers and Arrays(4).pptx
- E22: CSC3002 Final 2022 Spring.pdf

本册学习任务	原材料定位	要能独立完成的动作
对象、地址、指针与多级访问	L6 第 17-29、45-46 张；E22 Q1、Q6、Q7、Q15	区分指针对象与目标；逐层解引用
数组容量、有效长度、边界与步长	L6 第 35-38 张；E22 Q5、Q32	先写类型，再算元素位置和地址
前置、后置与组合表达式	L2 第 16 张；L6 第 42-44 张；E22 Q31	同步维护 <code>p</code> 、数组、表达式结果
左值与指针用途	L6 第 18、21、26 张；E22 Q25、Q26	区别对象表达式与临时计算结果

课件决定本册的概念范围；旧卷用于练题型和深度，不预测今年题目或分值。本册的解释、状态表和答案由教学推导得到；未找到官方答案册，因此不称为官方标准答案。保留的最小例子与一般练习属于自编教学补充；涉及原材料时逐项标明“课件例题”“原题代码”或“题意转述”。

当前 L1 第 2 张写的是闭卷，E22 第 1 页写的是可带一张双面 A4 的限制性开卷，两者不能混作今年规则。六册只覆盖 C++ 基础这一段，旧卷的数据结构、算法和部分继承多态内容仍需后续学习。

第一部分：先建立对象、地址和指针模型

1. 对象是什么？名字、类型和值分别是什么？

这里先研究最简单的对象：**一块具有类型、能存放相应值、在一定时间内存在的存储实体。** `int x = 5`；中有一个 `int` 对象，`x` 是访问它的名字，`5` 是当前存放的值。对象不等于名字：M2 的引用已经说明，同一对象可以通过不同名字访问。

```
int x = 5;
int y = 5;
```

这是两个不同对象，恰好都存放 5。把 `x` 改为 9，不会使 `y` 自动变为 9。因此，做指针题首先问“有几个对象”，再问它们的值。

观察角度	对 <code>x</code> 的回答	用来解决什么问题
名字	<code>x</code>	代码通过什么名字访问它
类型	<code>int</code>	能存什么值、允许什么操作
当前值	5	当前读取会得到什么
存储位置	运行时为它安排的位置	怎样间接找到同一对象

对象有存储位置，才有下面的“地址”。不要把“`x` 的值是 5”和“`x` 所在的位置”混为一谈。

2. 地址是什么，为什么要取地址？

地址是用来定位存储位置的信息。 想让另一个地方间接访问 `x`，可以先取得指向 `x` 的地址值。

```
int x = 5;
int* p = &x;
```

右边 `&x` 是一个表达式：这里 `&` 是一元取地址运算符，操作数为 `x`，结果类型为 `int*`，指向那个 `int` 对象。这个结果不是整数 5，也不是新建一个值为 5 的对象。

可以临时假设 `x` 的起始地址是 1000，帮助画图。但 **1000 是教学假设，不是程序的固定地址**；真实运行通常会显示不同的十六进制地址。C++ 指针的语义也不应被简化成“任意普通整数”。

```
位置 A: int 对象 x, 保存 5
&x: 指向位置 A 的地址值
```

取地址解决的是“找到对象”，读取 `x` 解决的是“取得对象当前的值”。这两步不同。

3. 指针是什么？指针变量自己也是对象

指针类型的值能够指向某个对象；指针变量是用来保存这种值的对象。 所以 `p` 自己也有类型、当前值和存储位置。

逐项读 `int* p = &x;`：`int` 说明目标对象类型；声明中的 `*` 说明 `p` 是指针；`p` 是变量名；`= &x` 用指向 `x` 的地址值初始化它；分号结束声明。

```
位置 B: 指针对象 p, 保存 指向位置 A 的值
      |
      v
位置 A: 整数对象 x, 保存 5
```

这里有两个对象：整数对象 `x` 和指针对象 `p`。`p` 保存的内容用于找到 `x`，但 `p` 不是 `x`。把信封上的投递地址改掉，和改收件人的房间内容，是两类操作；这个类比只帮助区分角色，不替代语言规则。

```
int x = 5;
int y = 8;
int* p = &x;
p = &y;
```

最后改变的是指针对象 `p` 保存的指向关系。`x` 仍为 5, `y` 仍为 8。声明 `int* p, q;` 也要留意: 只有 `p` 是指针, `q` 是 `int`。初学时分行声明更容易看清。

4. 解引用是什么? `*p` 为什么可以放在等号左边?

解引用是通过指针找到它所指向的对象, 并形成访问该对象的表达式。在表达式 `*p` 中, `*` 是一元间接访问运算符; 它与声明 `int* p` 中的声明符角色不同。

```
int x = 5;
int* p = &x;
int y = *p;
*p = 12;
```

第三行先读取 `p` 保存的指向关系, 找到 `x`, 再读取其值 5, 用来初始化独立对象 `y`。第四行通过 `*p` 找到同一个 `x`, 向它写入 12。最后 `x = 12`, `y = 5`, `p` 仍然指向 `x`。

因此 `*p` 不只是“算出一个数字”的口号: 它可以标识对象。在这个非 `const` 整数例子中, 它可用于读取, 也可作为赋值目标。

5. `x`、`&x`、`p`、`*p`、`&p` 一次分清

以下表格只针对 `int x = 5; int* p = &x;`, 并假定对象仍存活。

表达式	表达式类型	它在这里表示什么
<code>x</code>	<code>int</code>	名为 <code>x</code> 的整数对象; 读取时得到 5
<code>&x</code>	<code>int*</code>	指向 <code>x</code> 的地址值
<code>p</code>	<code>int*</code>	指针对象 <code>p</code> ; 读取时得到它保存的指针值
<code>*p</code>	<code>int</code>	<code>p</code> 指向的整数对象, 即 <code>x</code>
<code>&p</code>	<code>int**</code>	指向指针对象 <code>p</code> 自己的地址值

`int**` 读作“指向 `int` 指针的指针”。这里先理解 `&p` 指向 `p`, 而 `p` 指向 `x`, 不必马上展开复杂多级指针。

在此初始化后, `p == &x` 为真; `*p` 和 `x` 访问同一个对象。`p` 与 `&p` 既不是同一角色, 类型也不同。不要只凭打印出来都是一串地址字符就把它们当成一回事。

6. `p = ...` 和 `*p = ...`: 到底修改谁?

```
int x = 5, y = 8;
int* p = &x;
*p = 6;
p = &y;
*p = 9;
```

执行完哪一行	x	y	p 指向谁
初始化	5	8	x
*p = 6	6	8	x
p = &y	6	8	y
*p = 9	6	9	y

判断方法：左边是 `p`，写入的是指针对象；左边是 `*p`，写入的是它当时指向的目标。顺序必须保留，因为最后一行访问谁，取决于上一行有没有改变 `p`。

7. nullptr、未初始化和无效访问

空指针表示当前不指向一个可访问的目标对象。 `nullptr` 是 C++ 的空指针字面量。

```
int* p = nullptr;
if (p != nullptr) {
    std::cout << *p;
}
```

条件为假，因此不解引用。 `int* q`；则没有给局部指针初始化一个可用的值，不能把它当成自动指向某处的指针。

在本册普通对象访问中，解引用要求目标有效且对象仍存活。空指针、未初始化指针、越界位置和悬空指针都不能用来正常读写对象。“能编译”不保证“运行时访问合法”。有的错误属于未定义行为，不能承诺输出 0、报错或一定崩溃。

8. 指针和引用怎样接回 M2?

```
int x = 5, y = 8;
int& r = x;
int* p = &x;
r = y;
p = &y;
```

`r = y` 把 8 写进 `x`，没有重新绑定 `r`。 `p = &y` 则改变了指针保存的指向关系。最后 `r` 仍引用 `x`，`p` 指向 `y`。

方面	普通左值引用 <code>int&</code>	指针变量 <code>int*</code>
初始化后怎样访问目标	用引用名	用 <code>*p</code>
普通赋值的作用	给已绑定目标赋值	改变保存的指针值
是否有正常的空状态	不以空引用作为正常使用状态	可保存 <code>nullptr</code>
自己是不是普通指针对象	引用不是对象	指针变量是对象

指针作为参数也可以传值： `void set(int* q) { *q = 9; }` 在目标有效时能改目标，但 `q` 本身是独立指针参数。若函数内部只写 `q = nullptr`，不会把调用者的指针变量改成空。这里正好复用 M2 的独立对象模型。

第二部分：数组让模型扩展到连续的多个对象

9. 数组是什么，为什么需要它？

数组是一组同类型元素组成的对象，内建数组的元素连续存储。 如果要保存四个分数，数组使它们能用下标和循环统一处理，不必为四个变量写四份代码。

```
int a[4] = {10, 20, 30, 40};
```

`int` 是元素类型；`a` 是数组名；`[4]` 是元素数量，不是最大下标；花括号中给出初值。数组类型为 `int[4]`。元素 `a[0]` 到 `a[3]` 都是 `int` 对象。

下标：	0	1	2	3
元素：	10	20	30	40

局部 `int a[4];` 的元素未初始化，不能直接读取后假定为 0。写 `int a[4]{};` 可以把这些整数元素零初始化。

10. 连续存储是什么意思？步长为什么和类型有关？

假设 `sizeof(int) == 4`，并假设 `a[0]` 起始地址为 1000，则示意地址如下。

元素	示意起始地址	当前值
<code>a[0]</code>	1000	10
<code>a[1]</code>	1004	20
<code>a[2]</code>	1008	30
<code>a[3]</code>	1012	40

这是题设下的示意，不说明所有平台的 `int` 都是 4 字节。连续的意思是相邻元素之间没有额外的元素间空隙；步长为 `sizeof(元素类型)`。

指针加 1 移动一个目标类型的元素位置，不是无条件移动一个字节。 `int*` 的一步与 `char*` 的一步可能大小不同。`sizeof(char) == 1`，这里的 1 是 C++ 字节单位。

11. `a` 是数组，为什么又能给指针赋值？

```
int a[4] = {10, 20, 30, 40};
int* p = a;
```

在这个初始化语境中，数组表达式 `a` 转换为指向首元素的指针，等效于用 `&a[0]` 初始化 `p`。这个转换常被叫作数组到指针转换或“退化”。

数组没有因此变成指针变量。 `a` 仍是数组对象；`p` 才是保存指针值的独立变量。`p = p + 1;` 合法；`a = a + 1;` 不会成为合法的内置数组赋值。

这也解释 `sizeof(a)` 为什么量整个数组，而 `sizeof(p)` 量指针对象。在 `sizeof` 和一元取地址等语境中，不能套用“`a` 总是首元素指针”。数组转换规则可核对 C++ 工作草案：数组到指针转换。

12. `a`、`&a[0]` 和 `&a`：看起来同起点，类型不同

表达式或变量	类型	加 1 的含义
初始化后的 p	int*	前进一个 int 元素
&a[0]	int*	前进一个 int 元素
表达式 a 经数组转换后	int*	前进一个 int 元素
&a	int (*)[4]	前进一个完整的四元素数组大小

int (*)[4] 是“指向含四个 int 的数组的指针”。本册不要求背复杂声明，只要知道 &a 指向整个数组，和 &a[0] 的目标类型不同。

在上述 4 字节 int 的示意中，a + 1 对应第二个元素，&a + 1 对应整个数组之后的位置。它们的步长不同，不能仅凭起始地址的打印外观判断类型等价。

13. 下标访问怎样接到解引用？

对这个内建数组、合法下标 i：

```
a[i]      // 访问第 i 个元素
*(a + i)  // 访问同一个元素
```

例如 *(a + 2)：a 转为首元素指针，加 2 到第三个元素位置，解引用访问 a[2]，读取值为 30。a + 2 是指针表达式；*(a + 2) 是整数对象访问。丢掉 * 就改变了问题的层次。

```
int* p = a + 1;
p[1] = 99;
```

p[1] 相当于 *(p + 1)，从 a[1] 再前进一个元素，修改 a[2]。下标的起点由左边的指针当前值决定，不必总是数组的原始起点。

14. 边界：尾后指针可形成，但不能读取它

含四个元素的 a，合法元素下标为 0、1、2、3。a + 4 是尾后指针，可以用作循环终点；它不指向第五个元素对象，因此不能读取 *(a + 4)。

```
for (int* p = a; p != a + 4; ++p) {
    std::cout << *p << ' ';
}
```

每轮先比较是否到尾后，再访问有效元素，最后推进一步。到 a + 4 时循环停止，所以不解引用尾后位置。

形成 a + 5 已超出该数组允许的指针算术范围，不是等到解引用才第一次出问题。对同一数组内元素及其尾后位置做指针相减，得到元素位置差：(a + 3) - (a + 1) 为 2，不是字节差 8。不要拿不相关对象的指针随意做减法。见 [指针加减的边界规则](#)。

15. 数组参数为什么还需要长度？

```
int sum(const int a[], int n) {
    int total = 0;
    for (int i = 0; i < n; ++i) {
        total += a[i];
    }
    return total;
}
```

这里参数中的 `const int a[]` 调整为 `const int* a`，没有把整个数组按值复制进去。`n` 告诉函数允许访问多少个元素；调用者必须保证相应范围有效。`const` 限制函数通过这个参数写入元素。

不能在这种函数里用 `sizeof(a) / sizeof(a[0])` 推回调用者的元素数，因为这里的 `a` 已是指针参数。数组对象本身和数组参数的区别，会在 M10 的 `sizeof` 中再次出现。

第三部分：先讲普通 ++，再和指针组合

16. 普通前置和后置 ++ 是什么？

`++` 对本册普通整数表示增加 1。**前置 `++n` 提供修改后的结果；后置 `n++` 提供修改前的值，并使 `n` 增加 1。** 区别在“表达式提供什么”，不能仅说一个加得早、一个加得晚。

```
int n = 5;
int a = n++;
int b = ++n;
```

第一条初始化中，`n++` 提供旧值 5，`a = 5`；该完整语句结束时 `n = 6`。下一条 `++n` 使 `n = 7`，提供新结果，故 `b = 7`。

单独写 `n++`；与 `++n`；，若结果不再使用，在这里最终都让 `n` 增加 1。把它们放进更复杂表达式，才需要分别跟踪表达式结果和对象新状态。

17. 指针的 ++ 改变什么？

```
int a[3] = {10, 20, 30};
int* p = a;
++p;
```

这里增加的是指针位置：`p` 从指向 `a[0]` 变为指向 `a[1]`。三个元素的值完全没有改变。与之对照，`++(*p)`；修改当前目标元素的整数值，不移动指针。

因此每次见到 `++` 都问：**它的操作数是指针对象 `p`，还是整数对象 `*p`？** 这比先背四条口诀稳定得多。

18. 先解析结构：优先级不等于执行顺序

对本册用到的内建运算符，后置 `++` 比一元 `*` 结合得更紧；前置 `++` 与一元 `*` 处于同一优先级层次，按语法从右侧构成嵌套。括号可以明确改变分组。

原写法	先补上的括号	++ 操作的是谁
*p++	*(p++)	p
*++p	*(++p)	p
++*p	++(*p)	当前目标整数
(*p)++	(*p)++	当前目标整数

优先级回答“运算符怎样组成表达式”，不是“整个程序从左到右还是从右到左执行”。先补括号，再用每个操作的语义推演。本册把有副作用的表达式分成独立语句，避免靠含糊的执行顺序猜答案。

19. *p++：使用旧指针值访问，指针推进

以下每种演示都从全新的 `a = {10, 20, 30}`、`p = a` 开始。

```
int v = *p++;
```

先看结构 `*(p++)`。`p++` 提供 `p` 的旧指针值，它指向 `a[0]`；外层 `*` 通过该旧值访问 `a[0]`，所以 `v = 10`。该语句结束时，`p` 已推进到 `a[1]`。数组仍为 `{10, 20, 30}`。

用于理解结果的分步模型是：保存旧指针值 → 用旧指针读取目标 → 本语句结束时记录 `p` 的新位置。这是解释表达式值和最终状态的模型，不声称编译器必须按这几条机器指令实现。

20. *++p：推进指针，再通过新位置访问

```
int v = *++p;
```

结构为 `*(++p)`。内层 `++p` 使 `p` 指向 `a[1]`；外层 `*` 访问新的目标 `a[1]`，读取 20。最后 `v = 20`，`p` 指向 `a[1]`，数组未变。

这与上一节差别只有前置、后置位置：两者最终 `p` 都前进一步，但提供给解引用的指针值不同，所以 `v` 不同。

21. ++*p：增加目标整数，提供新结果

```
int v = ++*p;
```

结构为 `++(*p)`。内层 `*p` 表示 `a[0]`；前置 `++` 作用于该整数，把 10 改为 11，并提供修改后的结果。最后 `v = 11`，`p` 仍指向 `a[0]`，数组为 `{11, 20, 30}`。

这里没有任何一步要求给 `p` 加 1。只因为代码中出现星号和加号，不能就猜“指针移动”。

22. (*p)++：提供目标旧值，同时增加目标

```
int v = (*p)++;
```

括号把 `*p` 作为后置 `++` 的操作数。它访问 `a[0]`，向 `v` 提供旧值 10，并把目标整数增加为 11。最后 `v = 10`，`p` 仍指向 `a[0]`，数组为 `{11, 20, 30}`。

四种情况在相同起点下归纳如下。理解过程之后，再用此表复习。

语句右边	v	p 最后位置	a[0]	核心区别
*p++	10	a[1]	10	旧位置取值，移动 p
*++p	20	a[1]	10	新位置取值，移动 p
++*p	11	a[0]	11	改目标，提供新值
(*p)++	10	a[0]	11	改目标，提供旧值

前置、后置结果规则分别见 [前置自增](#) 与 [后置自增](#)。表格中的四行必须各自重置起点，不能误当成连续执行后的结果。

23. 连续执行时，每一行从上一行的状态开始

```
int a[4] = {3, 6, 9, 12};
int* p = a;
int u = *p++;
int v = ++*p;
int w = (*p)++;
int z = *++p;
```

执行后	本行结果	p 的位置	数组当前值
初始化	无	a[0]	3, 6, 9, 12
u 初始化	u = 3	a[1]	3, 6, 9, 12
v 初始化	v = 7	a[1]	3, 7, 9, 12
w 初始化	w = 7	a[1]	3, 8, 9, 12
z 初始化	z = 9	a[2]	3, 8, 9, 12

手推不要每行重新假设 p 在 a[0]。把“p 的位置”“数组内容”“表达式结果”分列，正是为了避免把三种状态混在一起。

24. 不要给非法表达式编造输出

```
int a[2] = {7, 8};
int* p = a + 2;
int v = *p;           // 错误示例：读取尾后位置
```

尾后指针可作为结束标记，不能当成对象读取。类似地，在 p 已指向尾后位置时再做 p++，会试图形成超范围位置。

另一类错误是同一完整表达式中对同一标量作未排序的冲突访问，例如 C++17 下的 `int y = n++ + n;`。这不是普通的“两种可能输出”，而是未定义行为。**遇到复杂自增题，先检查访问合法性和求值规则，再计算。**

25. 一个能独立运行的完整模型

```
#include <iostream>

int main() {
    int a[3] = {10, 20, 30};
    int* p = a;
    int first = *p++;
    ++*p;
    std::cout << first << ' ' << *p << ' '
               << a[0] << ' ' << a[1] << '\n';
}
```

`first` 从旧位置取得 10; `p` 移到 `a[1]`; `++*p` 把 `a[1]` 从 20 改为 21; 输出 10 21 10 21。本例把对象、地址、数组步长和自增完整接在一起。

回到 lecture notes: 补齐本册必须会的四个连接

A. allocated size 与 effective size: 有多少格, 与用了多少格

L6 第 36 张给出两个考试会用的术语。**allocated size** 是数组分配的元素数量; **effective size** 是程序当前有效使用的元素数量。后者需要程序自行维护, 内建数组没有自动保存这个业务计数。

```
int scores[6]{};
int used = 2;
scores[0] = 80;
scores[1] = 90;
```

数组有六个实际元素, 所以 **allocated size** 为 6; 当前只有前两个记录被当成有效分数, 所以 **effective size** 为 2。其余四个元素存在且已初始化为零, 但不因此自动成为四条有效成绩。

若再加入一条, 应先确认 `used < 6`, 写入 `scores[used]`, 再增加 `used`。**编译器通常不替内建下标检查边界, 也不阻止 `used` 被错误地加到 7。**E22 Q5 的错误正出在 “compiler makes sure” 这一句。数值范围由程序保证, 不是容量声明自动保证。

B. 二级指针: 不是多一颗星号就多猜一次

L6 第 45-46 张从 `i`、`pi`、`ppi` 扩展到三级指针。先停在两级, 建立够用的模型:

```
int i = 2022;
int* pi = &i;
int** ppi = &pi;
```

`pi` 是保存 `int` 对象位置的指针对象; `ppi` 是保存 `int*` 对象位置的指针对象。声明中的第二颗星号改变了目标类型, 不是在初始化时读取两次。

```
ppi 对象 --保存指向--> pi 对象 --保存指向--> i 对象
int**          int*          int
```

读取 `ppi` 得到 `pi` 的位置; `*ppi` 访问 `pi` 对象, 读取它得到 `i` 的位置; `**ppi` 访问 `i`, 读取它得到 2022。每去掉一层指针, 就沿箭头走一层。`&ppi` 则反过来取得 `ppi` 自己的位置, 类型为 `int***`。

因此, `*ppi = &other`; 改的是 `pi` 保存的指向, `**ppi = 8`; 改的是 `pi` 当时指向的整数。先指出赋值左边表示哪个对象, 再写新值。

C. L6 第 28 张的 swap：交换指针，还是交换目标？

下面是按课件机制整理的独立教学版。每种调用都从 `int a=1,b=2; int* p=&a; int* q=&b;` 重新开始。

```
void localSwap(int* x, int* y) {
    int* tmp = x;
    x = y;
    y = tmp;
}
void valueSwap(int* x, int* y) {
    int tmp = *x;
    *x = *y;
    *y = tmp;
}
void pointerSwap(int*& x, int*& y) {
    int* tmp = x;
    x = y;
    y = tmp;
}
```

第一种 `x`、`y` 是独立的指针参数，交换的只是参数副本；调用结束后 `p`、`q` 仍分别指向 `a`、`b`，`a=1`、`b=2`。第二种仍复制指针值，但写入 `*x`、`*y`，所以修改共同目标，`a=2`、`b=1`，`p`、`q` 不动。

第三种中的 `int*&` 是“对 `int*` 对象的引用”：`x` 绑定 `p`，`y` 绑定 `q`。交换 `x`、`y` 就直接交换调用者的指针值。调用后 `p` 指向 `b`，`q` 指向 `a`，而 `a=1`、`b=2`。它补齐了 M2 的问题：**要修改外部指针对象，借用的是那根指针；要修改外部整数，访问的是指针的目标。**

D. 左值是什么？先分清对象，再讨论能否赋值

L6 第 18 张用“能放在赋值左边”帮助入门，Q26 也使用这个课程模型。更准确地说，本册的左值表达式可以标识对象；**其中只有满足条件的可修改左值才可用于普通赋值。**例如 `const int` 对象对应的表达式是左值，但不能给它赋新值。

对有效的 `int* p`，`p` 表示指针对象，`*p` 表示目标整数，都是左值；`p+1` 产生计算出的新指针值，`&p` 产生地址值，都不是左值。不要把“指针类型”与“左值类别”混成一个标签。

课件 L6 第 25 张把 `double*` 强转为 `int*` 后读取，展示了某次运行结果。这种错误类型访问不能当成可移植的数值转换：要把 2.1 转成整数，应讨论数值转换；不能通过换一种指针类型读它来推算固定输出。本册不运行该错误片段。

练习：先自行推演，再看后面的解析

题 1：五个表达式

`int x = 5; int* p = &x;`。分别说明 `x`、`&x`、`p`、`*p`、`&p` 的类型和角色。执行 `*p = 8;` 后哪个对象改变？

题 2：移动指针还是修改目标？

```
int a = 2, b = 7;
int* p = &a;
int* q = p;
p = &b;
*q = 9;
```

最后 `a`、`b` 分别是多少？`p` 和 `q` 分别指向谁？

题 3：数组步长

假设 `int` 为 4 字节，`int a[4] = {2,4,6,8};` 首元素地址为 1000。`a + 3` 的示意地址是什么？`*(a + 3)` 是多少？`(a + 3) - a` 是多少？

题 4：独立比较四种表达式

每次都重新建立 `int a[3] = {4,8,12}; int* p = a;`。分别用 `*p++`、`*++p`、`++*p`、`(*p)++` 初始化 `v`，给出 `v`、`p` 位置和数组内容。

题 5：连续手推

```
int a[4] = {2, 4, 6, 8};
int* p = a;
int x = *p++;
int y = (*p)++;
int z = *++p;
int w = ++*p;
```

给出 `x`、`y`、`z`、`w` 和最终数组。

题 6：边界判断

对 `int a[3] = {1,2,3};`，分别判断 `a + 3`、读取 `*(a + 3)`、形成 `a + 4`、读取 `*(a + 2)`。不要运行错误示例。

题 7：函数里的指针参数

```
void change(int* q) {
    *q = 20;
    q = nullptr;
}
int a = 5;
int* p = &a;
change(p);
```

最后 `a` 和 `p` 是什么状态？为什么函数既能改 `a`，又没把 `p` 改为空？

练习详细解析

题 1 解析：区分目标对象和指针对象

x 类型 `int`，表示整数对象；`&x` 类型 `int*`，是指向 x 的地址值； p 类型 `int*`，是指针对象，保存指向 x 的值；`*p` 类型 `int`，表示 x ；`&p` 类型 `int**`，指向指针对象 p 。

赋值左边是 `*p`，因此修改目标 x ，使 $x = 8$ ； p 的指向未改变。若误答“ p 变成 8”，就是没有区分指针保存的地址和目标保存的整数。

题 2 解析：复制指针值没有绑定两个指针变量

$q = p$ 的初始化让 q 也指向 a ，但 p 和 q 是两个独立指针对象。之后 $p = \&b$ 只移动 p ，不改变 q 。`*q = 9` 仍修改 a 。

结果： $a = 9$ ， $b = 7$ ， p 指向 b ， q 指向 a 。可迁移规则是：**复制指针值会让两个指针指向同一目标，不会让两个指针变量变成同一对象。**

题 3 解析：地址差和元素差不能混

$a + 3$ 前进三个 `int` 元素，在给定假设下是 $1000 + 3 \times 4 = 1012$ 。解引用读取 `a[3]`，得到 8。指针相减的结果是位置差 3，类型为 `std::ptrdiff_t`，不是字节差 12。

题 4 解析：每行都重置状态

表达式	v	p 指向	最终数组
<code>*p++</code>	4	<code>a[1]</code>	4,8,12
<code>*++p</code>	8	<code>a[1]</code>	4,8,12
<code>++*p</code>	5	<code>a[0]</code>	5,8,12
<code>(*p)++</code>	4	<code>a[0]</code>	5,8,12

前两行 `++` 操作 p ，后两行操作 `*p`。每组中前置提供新结果、后置提供旧值。先确认修改对象，再决定用新值还是旧值。

题 5 解析：保留上一行留下的状态

第一行赋给 x 旧目标值 2， p 移到 `a[1]`。第二行 y 取得 `a[1]` 的旧值 4，`a[1]` 变为 5， p 不动。第三行 p 移到 `a[2]`， $z = 6$ 。第四行 `a[2]` 变为 7， $w = 7$ 。

最终 $x = 2$ ， $y = 4$ ， $z = 6$ ， $w = 7$ ，数组 `{2,5,7,8}`， p 指向 `a[2]`。若 z 误算为 8，通常是错误地把第二行也当成移动 p 。

题 6 解析：形成位置和访问对象分别判断

$a + 3$ 合法形成尾后指针；读取 `*(a + 3)` 非法，因为不存在 `a[3]` 元素；形成 $a + 4$ 已越过允许的指针算术范围；读取 `*(a + 2)` 合法，结果为 3。非法项没有可靠的固定运行输出。

题 7 解析：参数是指针值的副本，目标仍相同

调用时 q 用 p 的指针值初始化，因此都指向 a 。`*q = 20` 修改共同目标 a 。`q = nullptr` 只改独立参数 q ，不改 p 。最后 $a = 20$ ， p 仍指向 a 。

这将 M2 和 M3 直接连起来：传值并不表示“通过参数绝不可能影响外部”，它表示参数对象独立；若参数保存了外部对象的位置，仍可经该位置访问外部对象。

学完这一册后的衔接

能独立回答“表达式指谁、++ 改谁、下标是否有效”后，进入 M6：把数组元素类型换为 char，再加上 `\0` 结束规则。需要复习时优先回到对象关系和状态表，不必从四条表达式口诀重新开始。

旧卷训练：原题与详细解析

以下题意用中文转述；代码、数字和选项对应关系按 E22 核对。答案为本讲义推导。

E1. Q1、Q6、Q7、Q5、Q26：短判断也需要模型

Q1（第 2 页）用“变量是有名字的地址、指针保存另一个变量地址”描述两者。按课程的普通对象模型，预期判 True；更精确的理解是变量提供对对象或引用的命名，指针变量是存储指针值的对象。不要把这个入门描述扩大成所有 C++ 变量的严格定义。

Q6（第 2 页）的原声明是：

```
int* p1, p2;
int *p3, p4;
```

题面说 p1、p3 为指针，p2、p4 为整数，答案 True。星号属于各个声明符；空格换位置不会把 p2、p4 变成指针。

Q7（第 2 页）讨论 *&x 与 x、&*p 与 p。对本册有效的普通对象及内建指针操作，按课程意图判 True：前者取到位置再访问原对象，后者由目标取得对应位置。本册不把它用作对无效指针做任意操作的许可。

Q5（第 2 页）答案 False。allocated/effective 两个名称没有问题，错误是声称编译器保证 effective size 永不超过 allocated size。内建数组越界需要程序避免。

Q26（第 6 页）的选项依次是 p、(p+1)、&p、*p，问哪些不能是左值。普通指针语境下选 b、c。p 是可修改的指针变量，*p 在目标为可修改 int 时可写；另外两项只是计算结果。const 左值提醒说明题面定义经过了简化，不能背成“不能赋值就一定不是左值”。

E2. Q15：给定地址的二级指针

原题（第 3-4 页）：

```
int i = 2022, *pi = &i, **ppi = &pi;
```

给定 i、pi、ppi 自己的地址分别为 0x5190、0x5198、0x51A0，问 **ppi 和 *ppi 的值。

对象	自己的位置	保存的值
i	0x5190	2022
pi	0x5198	0x5190
ppi	0x51A0	0x5198

第一层 *ppi 访问 pi，所以其读取值为 0x5190；第二层 **ppi 访问 i，读取值为 2022。题目问的顺序是 **ppi 在前，因此答案为 **2022、0x5190，选 d**。若填 0x51A0，是把 ppi 自己的位置与其保存内容混淆。

E3. Q31：连续执行四种表达式，保留输出格式

原题代码（第 7-8 页）：


```

#include <iostream>
using namespace std;
int main(void) {
    int arr[] = {0, 5, 1, 9};
    int* p = arr;
    int a = *++p;
    int b = ++*p;
    int c = *p++;
    int d = (*p)++;
    cout << a << b << c << d << endl;
    cout << arr[0] << arr[1] << arr[2] << arr[3] << endl;
    return 0;
}

```

语句执行后	本次结果	p 指向	arr 的当前内容
初始化	无	arr[0]	0,5,1,9
a = *++p	a=5	arr[1]	0,5,1,9
b = ++*p	b=6	arr[1]	0,6,1,9
c = *p++	c=6	arr[2]	0,6,1,9
d = (*p)++	d=1	arr[2]	0,6,2,9

a 那行移动 p 后读取新目标；b 那行只把目标 5 加为 6；c 那行从旧目标读 6，再让 p 前进一步；d 那行取目标旧值 1，并把该元素改为 2。两行 cout 都没有空格或逗号，所以实际输出是：

```

5661
0629

```

第二行开头的 0 必须保留，它来自 arr[0] 的单独输出，不是把整行当作整数去掉前导零。本题原答案已经讲过时，复述 5661 不算独立掌握；请回到自编连续题，改变数组初值后重新手推。

E4. Q32：先标地址项和数值项，再填十六进制

原声明（第 8 页）：

```
double doubleArray[] = {1, 3, 5, 7, 9, 11, 13, 15, 17, 19};
```

题设 double 占 8 字节，首地址为 0x5190，要求用四位十六进制写结果。原卷第一项 doubleArray 已预填 5190，下面另有 12 个空。表中保留全部 13 项方便对照。

表达式	先读出它表示什么	按题设填写
<code>doubleArray</code>	首元素位置，原卷已给	5190
<code>&doubleArray</code>	整个数组的起始位置	5190
<code>*doubleArray</code>	首元素数值 1	0001
<code>doubleArray+1</code>	前进一个 double: 加 8	5198
<code>&doubleArray+1</code>	前进整个数组: 加 80=0x50	51E0
<code>*doubleArray+1</code>	先读数值 1, 再加 1	0002
<code>doubleArray[2]</code>	第三个元素数值 5	0005
<code>&doubleArray[2]</code>	首地址加 2×8	51A0
<code>doubleArray+8</code>	首地址加 8×8	51D0
<code>*doubleArray+8</code>	数值 1 加 8	0009
<code>*(doubleArray+8)</code>	第九个元素数值 17	0011
<code>doubleArray[9]</code>	第十个元素数值 19	0013
<code>&doubleArray[9]</code>	首地址加 9×8	51D8

关键是 `*doubleArray+8` 按 `(*doubleArray)+8` 分组，算的是 1+8；`*(doubleArray+8)` 则先找第九个元素，再读 17。`&doubleArray` 的类型是指向十元素 double 数组的指针，步长是 10×8；普通 `doubleArray` 转换后的指针步长才是 8。

这里把数值 1、17 等按题面约定记为十六进制整数形式，不是在打印 double 的 IEEE 二进制位模式。也不能直接写 `cout << hex << doubleArray[8]` 就期待得到 0011：hex 主要设置整数进制，题面表格是在规定答案的记法。

E5. Q25: 指针用途怎样与代码对应?

原题（第 6 页）列出动态对象的地址、运行期间申请新存储、紧凑引用大结构、记录链式关系四类用途。结合 L6 第 21、26 张，按课程意图四项均属于指针相关用途。申请存储由 new 等机制完成，指针保存结果并让程序访问它；这不表示一个裸指针声明自身会申请目标对象，也不表示程序能任意指定堆的实际地址。

做完本册应能在新数字下重建对象图和状态表。下一册把同一数组模型换为 char，再加上零结尾规则。